Remaining Phases — PR7 through PR13

Context for the freelancer: PR1 through PR6 are complete. Those phases delivered the core clock wheel system — database schema, schedule conflict detection, the hour timeline engine, validation and logging, the frontend slot editor, and full automated test coverage. PR8 (Liquidsoap broadcast layer alignment) is also complete. This document covers only the remaining phases: PR7 and PR9 through PR13. These are the enterprise enhancements that take the system to professional broadcast automation grade. The existing codebase, APIs, and data model are the foundation — none of this work requires changing anything already built.

Remaining Work — PR7 and PR9 through PR13

Six phases remaining. All build on top of the completed PR1–PR8 foundation.

PR	Name	Priority	Size	Description
PR7	**Schedule Page — Live Clock Wheel Tab**	High	Medium	New tab on schedule page showing current wheel on air with track names, upcoming queued content, and future wheel playlists. Read-only broadcast operations display. Zero backend changes.
PR9	**Separation Rules + Burn Rate Protection**	High	Medium	Artist/title/category/tempo separation with daypart-aware overrides. Playlist burn rate protection stops small categories dominating rotation.
PR10	**Daypart Clock Inheritance**	High	Medium	ClockTemplate and ClockInstance entities. Assign templates to dayparts. Template changes propagate to all assigned hours automatically.
PR11	**Audit Event Database**	High	Small	Persistent clock_wheel_events table storing every scheduling decision. Foundation for PR12 analytics dashboard.
PR12	**Preview Simulator + Analytics Dashboard**	Medium	Medium	Next-hour preview endpoint. Rotation analytics dashboard. Fill strategy config. Drift reporting.
PR13	**Emergency Override**	Optional	Small	is_emergency boolean on StationSchedule. Immediate preemption of active clock wheel. No numeric priority stack.

PR7 — Schedule Page: Live Clock Wheel Tab

Broadcast operations display — current wheel on air plus upcoming content

PR 7 · High · Medium

Schedule Page — Live Clock Wheel Tab

A new tab on the existing schedule page giving operators a live view of the current clock wheel on air including actual track names and artists, upcoming queued content for the current hour, and pre-generated content lists for future scheduled wheels. Read-only. Zero backend changes — reads from APIs already built in PR2–PR5.

What makes this more than just a visual: The clock face shows the shape of the hour. The content layer below it shows the actual songs and IDs the system has generated — not just slot type labels. An operator can see hours ahead what is lined up without opening a single config page.

What The Tab Displays

Element	Description
Live Clock Face	Circular clock face showing the current hour's wheel. A clock hand rotates in real time in station timezone. The currently active slot arc is highlighted. Color coded by slot type — music, ID, promo, ad each a distinct color.
Now Playing	Prominently displayed: track title, artist name, slot type. Time the slot started, estimated end time, seconds remaining until the next slot fires.
Current Hour Slots	List of remaining slots in the current hour. Each row shows target time, slot type, and the specific track or ID the system has already queued for that slot — not just the type label.
Next Hour Preview	The wheel assigned to the next hour shown alongside with its full queued content list — track titles and artists already generated by the engine for each slot so the operator can see what is coming.
Future Wheels	Scrollable list of clock wheels scheduled across the rest of the day. Each shows daypart name, start time, and the pre-generated content the system has planned — operators can see hours ahead what is lined up.
Conflict Warning	Visual alert when a scheduled playlist or streamer is about to override the clock wheel — shows which schedule is taking over and at what time so the operator is never surprised.

Implementation Notes

- New tab component on the existing schedule page alongside current tabs.
- Clock hand rotation is pure CSS animation driven by current time in station timezone — no polling needed for the hand itself.

- Now playing and slot data polled from existing PR3/PR4 queue APIs on a short interval (suggested every 10 seconds).
- Queued content for upcoming slots pulled from the GET .../preview endpoint built in PR5.
- Future wheel content lists drawn from queue data and the clock_wheel_events table built in PR11.
- Conflict warning reads from PR2 conflict detection data already in the schedule API.
- Zero backend changes — reads entirely from APIs already built in PR2, PR3, PR4, PR5, PR11.
- Clock wheel editor page form stays completely unchanged — this tab is monitoring only.

Done When

Schedule page has a Clock Wheels tab. Live clock hand rotates correctly in station timezone. Now playing shows actual track title and artist. Current hour slots show specific queued content not just slot type. Next hour and future wheels show pre-generated track lists. Conflict warning fires when a playlist or streamer is about to take over.

PR9 — Separation Rules + Burn Rate Protection

What must not play yet — and what must not dominate rotation

PR 9 · High · Medium

Separation Rules + Burn Rate Protection

Two related candidate filtering features that live in the same service layer. Separation rules prevent the same artist or song repeating too soon. Burn rate protection stops small playlists exhausting themselves and dominating rotation.

Every professional music scheduling system — RCS NexGen, MusicMaster, Selector — enforces both of these. Without separation rules the same artist can play twice in one hour. Without burn rate protection a 10-song category cycles all day while the rest of the library sits unused. These are what separate a broadcast automation system from a sophisticated playlist rotator.

Part 1 — Separation Rules

Rule Type	Description	Configuration
Artist separation	Minimum minutes before same artist plays again	Configurable per wheel
Title separation	Minimum hours/days before same song plays again	Common values: 3h, 24h, 7 days
Category separation	Minimum gap between tracks from same category/playlist	Per-category config
Tempo separation	Avoid consecutive fast or slow tracks	Optional, tag-based
Gender separation	Alternate male/female/group vocal where possible	Optional, tag-based
Decade separation	Avoid clustering tracks from same era	Optional, tag-based

Daypart-Aware Separation Overrides

'No artist repeat within 120 minutes' is correct for morning drive with deep inventory. At 3am with a sparse overnight rotation the same rule leaves the scheduler with no valid candidates, causing constant AutoDJ fallback. Add a **separation_override** field at the clock template level (PR10) so overnight and weekend clocks can relax rules relative to primetime. Relaxation cascade when candidates exhausted: decade → gender → tempo → category → artist → title. Log which rule was relaxed in the PR11 audit table.

Part 2 — Playlist Burn Rate Protection

Separation rules stop the same song repeating too soon. They do not stop the system burning through a small category too fast. If you have 10 songs in 'Power Current' and the wheel draws from it every hour, listeners hear those same 10 songs cycling all day while a 500-song library sits untouched. Burn rate protection tracks draw frequency per category and deprioritises exhausted pools — nudging the scheduler toward deeper catalog.

- Track draw count per playlist/category per configurable time window (e.g. last 24 hours).
- When draw count exceeds configured threshold, deprioritise that category — prefer less-used pools.
- Configurable threshold per playlist — a 10-song category has a lower threshold than a 500-song library.
- Log burn rate warnings in clock_wheel_events when a category is being deprioritised.
- If all categories hit threshold, fall through to least-burned pool — never block playback.
- Surface burn rate status in the PR12 analytics dashboard so operators can see which categories need more content added.

New Service

Both features live in **App\Radio\AutoDJ\SeparationRulesChecker**. Integrated into HourTimeline candidate filtering from PR3 — runs after fit-to-window filter, before algorithm selection. Uses **StationQueueRepository::getRecentlyPlayedByTimeRange()** with extended lookback beyond current hour.

Done When

Same artist does not repeat within configured separation window. Overnight clock relaxes separation rules correctly per daypart override. Relaxation cascade fires in correct order and logs which rule was relaxed. Small playlist does not dominate rotation when larger pools are available. Burn rate warnings appear in clock_wheel_events. Analytics dashboard shows category draw rates. Playback never stalls — always falls through to least-burned pool.

PR10 — Daypart Clock Inheritance

The single highest-value workflow feature for real program director operations

PR 10 · High · Medium

Daypart Clock Inheritance

Real stations build 6–8 clock templates and assign them to dayparts — not 168 individual hours. A change to one template propagates to every hour that uses it automatically. Without this, managing a full week of programming is impractical at scale.

The Problem Without This

A station running a Monday–Friday morning drive clock for 52 weeks needs to build and maintain one clock — not 260 individual configurations. When the PD wants to add a promo sweep at 45 minutes into every morning drive hour, they change one template. Without daypart inheritance they open 260 individual hour configs and make the same edit 260 times.

New Entities

Entity	Description
ClockTemplate	Master clock definition. Contains all slots, types, algorithms, separation overrides. Belongs to a station. Editing the template propagates to all instances that have not been individually overridden.
ClockInstance	A specific hour assignment. References a ClockTemplate. Instance-level slot overrides are stored separately and take precedence over the template for that slot only.
Daypart	Named recurring time block. Example: Morning Drive Mon–Fri 6am–10am. Maps a ClockTemplate to a recurring schedule window. Creating a daypart auto-generates ClockInstances for every hour in that window.

Behavior Rules

- Changing a slot in a template propagates to all instances that have not locally overridden that slot.
- An instance-level override takes precedence over the template for that slot only — all other slots still inherit.
- Deleting a template requires confirmation if active instances reference it.
- Copying a template creates a fully independent new template — not a live shared reference.
- Export a template to another station creates an independent copy — no live cross-station sharing.

Standard Daypart Examples

Daypart	Typical Window

Morning Drive	Mon–Fri 5:00am – 10:00am
Midday	Mon–Fri 10:00am – 3:00pm
Afternoon Drive	Mon–Fri 3:00pm – 7:00pm
Evening	Mon–Fri 7:00pm – 12:00am
Overnight	Daily 12:00am – 5:00am
Weekend	Sat–Sun all day

Done When

Changing a slot in a template propagates to all non-overridden instances. An instance with a local override retains it after template change. Deleting a template with active instances prompts confirmation. Daypart assignment auto-generates instances for every hour in the window.

PR11 — Audit Event Database

Persistent record of every scheduling decision — foundation for PR12 analytics

PR 11 · High · Small

Audit Event Database

A new database table that stores every clock wheel scheduling decision permanently. Structured logs disappear on rotation. This table makes every action auditable, queryable, and available for the PR12 analytics dashboard.

New Table: clock_wheel_events

Column	Description
timestamp	Unix timestamp of the scheduling decision
wheel_id	FK to station_clock_wheels
slot_id	FK to station_clock_wheel_slots
media_id	FK to station_media — nullable if AutoDJ fallback
expected_time	Target position_seconds for this slot
actual_time	Actual T (seconds into hour) when slot fired
drift_seconds	actual_time minus expected_time — measures scheduling accuracy
anchor_type	Snapshot of slot anchor_type (soft/hard) at execution time
fallback_reason	NULL if resolved normally. Reason string if AutoDJ took over.
separation_relaxed	Boolean — true if any separation rule was relaxed to find a candidate
burn_rate_warning	Boolean — true if a category was deprioritised due to burn rate

Done When

A row is written to clock_wheel_events on every scheduling decision. fallback_reason populated whenever AutoDJ takes over. drift_seconds calculated and stored on every soft anchor slot. burn_rate_warning and separation_relaxed flags populated correctly.

PR12 — Preview Simulator + Analytics Dashboard

See what the next hour will play. See how accurately the system has been running.

PR 12 · Medium · Medium

Preview Simulator + Analytics Dashboard

Two features built on the PR11 audit table. The preview simulator shows operators what the next hour will play before it happens. The analytics dashboard shows how accurately the system has been running across any time period.

Preview Simulator

Scoped to next hour only: A week or day preview is compute-heavy and unreliable because media availability and separation state change constantly. The only preview that genuinely matters is **'what will the next hour play given current queue state?'** That is what this delivers.

GET .../preview returns a projected slot sequence for the next hour:

- Slot time (mm:ss) and anchor type (soft/hard)
- Projected track title and artist — or 'AutoDJ fallback' if nothing resolves
- Estimated drift from target position based on current queue state
- Any separation rules that would exclude candidates from this slot
- Any burn rate warnings for categories targeted by this slot

Analytics Dashboard

Built directly on the **clock_wheel_events** table from PR11:

Metric	Description
Clock Accuracy	Percentage of slots that fired within soft tolerance of target position_seconds
Missed Hard Anchors	Count of hard anchors crossed — should always be zero
AutoDJ Fallbacks	Count and reason breakdown of AutoDJ takeovers per time period
Average Drift	Mean drift_seconds across all soft-anchor slots
Slot Fire Rate	How often each slot type actually fires versus being skipped
Burn Rate by Category	Draw frequency per playlist/category — shows which categories need more content
Separation Relaxations	How often and which rules were relaxed due to candidate exhaustion
Top Fallback Reasons	Ranked list: no candidates, separation exhausted, window too small, burn rate hit

Fill Strategy Configuration

Add a wheel-level **fill_strategy** enum: **conservative** (never select a track that would overrun the next anchor) or **aggressive** (allow the shortest available track even if it slightly overruns a soft anchor). Hard anchors always override fill strategy regardless of wheel setting. Covers 90% of real-world cases without per-slot configuration complexity.

Done When

Preview endpoint returns an accurate next-hour projection with track titles, drift estimates, and separation/burn warnings. Analytics dashboard displays all metrics above. fill_strategy configurable per wheel and respected at runtime.

PR13 — Emergency Override (Optional)

Immediate preemption for breaking news, weather emergencies, or network joins

PR 13 · Optional · Small

Emergency Override

A simple boolean flag on StationSchedule that immediately preempts whatever the clock wheel is doing. Intentionally kept minimal — one flag, clear semantics, no complexity.

Why a boolean and not a priority stack: A full numeric priority system (100 = Emergency, 50 = Live DJ, 10 = Clock Wheel) is how enterprise systems like WideOrbit handle it — and it is also where they become brittle and hard to reason about. For AzuraCast: **is_emergency BOOLEAN DEFAULT FALSE** on StationSchedule. When true, that schedule immediately preempts the active clock wheel slot. One flag, auditable, sufficient. Revisit a priority stack only if operators demonstrate a need for more than two levels.

Implementation

- Add is_emergency BOOLEAN DEFAULT FALSE to StationSchedule entity and migration.
- In ClockWheelScheduler::buildFromClockWheel(), check is_emergency on any competing schedule before returning early.
- Emergency schedules bypass conflict detection — always allowed to override regardless of schedule window.
- Log emergency preemption as fallback_reason = 'emergency_override' in clock_wheel_events.
- Surface is_emergency as a checkbox in CreateEventModal with a clear warning label.

Done When

A schedule marked is_emergency immediately preempts an active clock wheel slot. Preemption logged in clock_wheel_events with fallback_reason = 'emergency_override'. Non-emergency schedules continue through normal conflict detection unchanged.

Dependency Order

What must land before what

PR7 and PR9 can be worked in parallel immediately — neither blocks the other. PR10 is independent and can start any time. PR11 must land before PR12. PR13 is optional and independent.

PR	Name	Depends On	Notes
PR7	Schedule Page Live Tab	PR5 (complete)	Frontend only — no backend changes needed
PR9	Separation Rules + Burn Rate	PR3, PR6 (complete)	Extends candidate filtering built in PR3
PR10	Daypart Inheritance	PR1, PR6 (complete)	New entities, independent of engine logic
PR11	Audit Event Database	PR4, PR6 (complete)	Required before PR12 can be built
PR12	Preview + Analytics	PR11	Needs audit table — build after PR11 lands
PR13	Emergency Override (Optional)	PR6 (complete)	Independent — can land any time after PR6

Acceptance Criteria

What must be true before each PR is considered complete

PR	Acceptance Criterion	Verification
PR7	Schedule page Clock Wheels tab shows live track title, artist, rotating clock hand, upcoming queued content, future wheel playlists, and conflict warnings	Manual QA + polling interval test
PR9	Same artist does not repeat within configured separation window. Small playlist does not dominate rotation when larger pools available. Relaxation cascade logs correctly.	Unit tests + rotation review
PR10	Template change propagates to all non-overridden instances. Instance override retained after template change. Daypart auto-generates instances for every hour in window.	Inheritance unit tests
PR11	clock_wheel_events row written on every scheduling decision. drift_seconds, burn_rate_warning, and separation_relaxed populated correctly.	Audit table integration test

PR12	Preview endpoint returns accurate next-hour projection with titles and drift estimates. Analytics dashboard displays all metrics. fill_strategy respected at runtime.	Integration test + dashboard QA
PR13	is_emergency schedule immediately preempts active clock wheel. Logged as emergency_override in clock_wheel_events.	Scheduler unit test

Effort Estimates

Rough sizing — not sprint commitments

PR	Name	Size	Notes
PR7	Schedule Page Live Tab	Medium	Frontend only — CSS animation + API polling
PR9	Separation Rules + Burn Rate	Medium	New service layer on existing candidate filtering
PR10	Daypart Clock Inheritance	Medium	New entities + UI — highest PD workflow impact
PR11	Audit Event Database	Small	New table + write hooks in existing scheduler
PR12	Preview + Analytics	Medium	Endpoint + dashboard — needs PR11 first
PR13	Emergency Override	Small	One boolean field + scheduler guard check

What Not To Build

Architectural commitments — do not change without documented alternative

Do Not Build	Reason
Hard-cutting music mid-song	Always select a track that completes before the next hard anchor, or defer to AutoDJ. Never truncate.
Wrapping slots in the same hour	If all slots have passed, fall through to AutoDJ. Wrap behavior is a future configurable enhancement — not v1.
Numeric priority stacks	Use is_emergency boolean in PR13. A full priority number system is complex and opaque for no additional benefit at this scale.
Multi-station live clock sharing	Shared live clocks across stations cause subtle cross-station bugs. Use export/import as independent copies instead.

Preemptive Liquidsoap changes	PR8 is complete. Do not revisit Liquidsoap unless PR12 analytics show a real drift problem in production.
Sub-second synchronization	Internet radio delivery latency makes sub-second precision unmeasurable and unachievable end-to-end.

Notes

PR1–PR8 are complete. This document covers only the remaining phases: PR7 and PR9–PR13. The core architecture of the completed system — fit-to-window selection, no hard cuts, calendar authority over clock wheels, AutoDJ fallback — must not be changed as part of any of these remaining phases. All new features build on top of that foundation.

> *Architecture decisions marked with warning boxes represent deliberate choices that differ from common enterprise patterns. Rationale is provided inline. Do not override without explicit approval and a documented alternative design.*